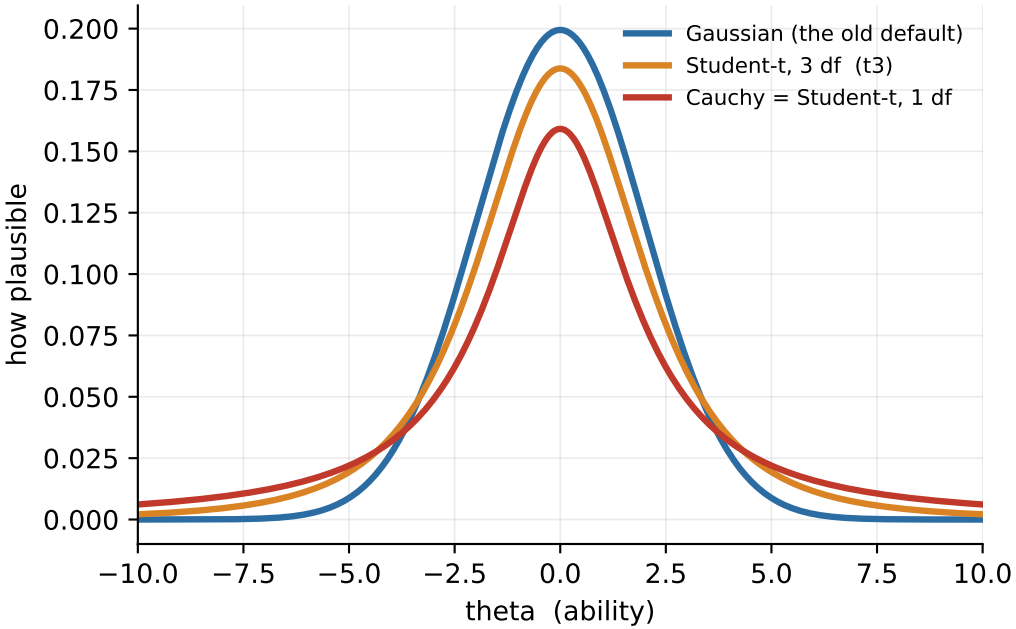
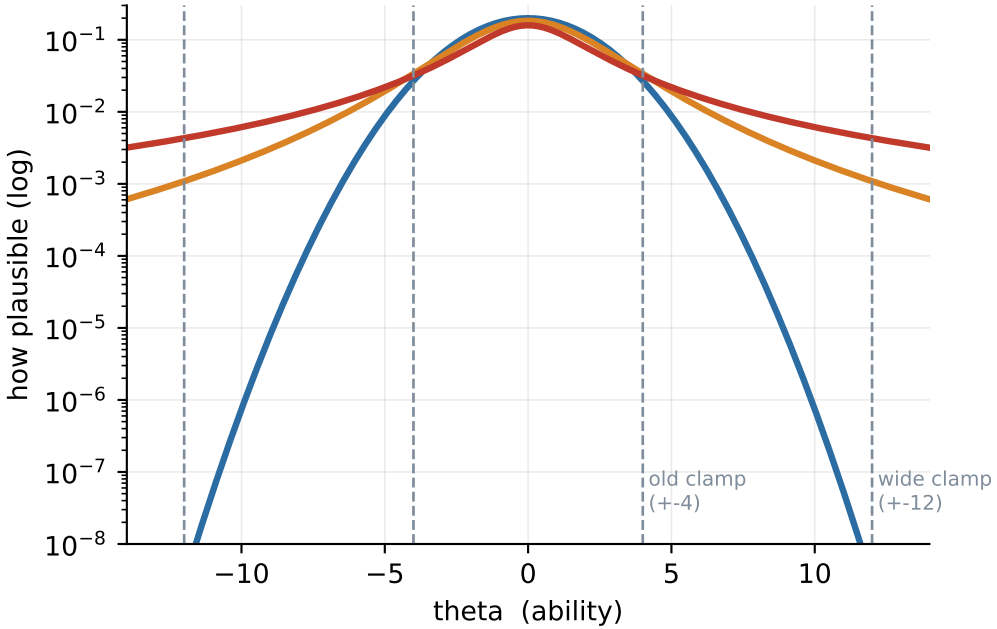
What is an ability prior?

Every subject gets one number, theta, for how good it is (higher = better). Before we see any of its answers, we still have to guess a value. The prior is that guess, drawn as a hill: the higher the hill at a value, the more plausible that value is BEFORE seeing any labels.

A. The three hills (same width, different tails)



B. Same picture, log scale: zoom into the tails



How to read it

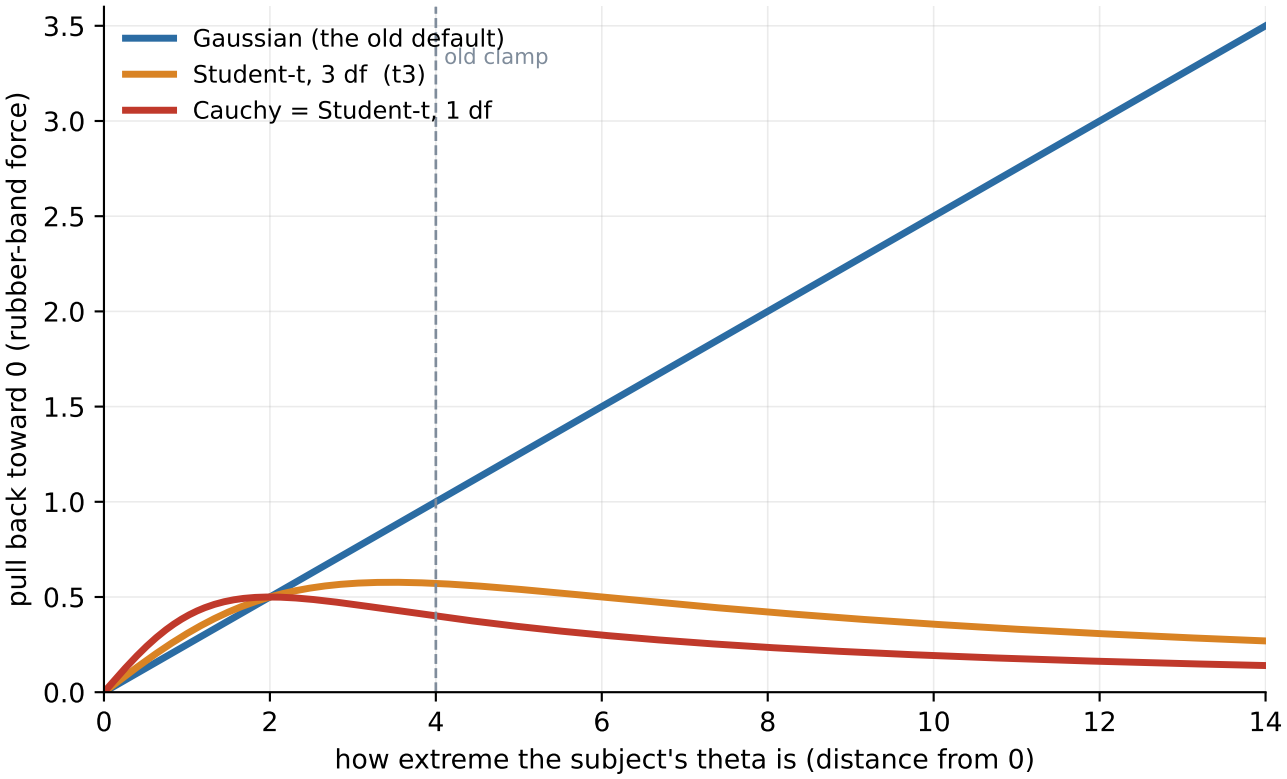
- Near the middle all three look alike. They only disagree about EXTREME subjects.
- Gaussian: extreme values are very unlikely. At theta = 8 it is about 3,000 times less plausible than at 0.
- Student-t / Cauchy: extreme values are rare but possible. They have 'heavy tails'.
- Cauchy has the heaviest tails, t3 is in between, Gaussian has the lightest.
- The dashed lines are the CLAMP: a hard wall. The solver is not allowed to put theta beyond it, no matter what the answers say. 'Old clamp' = +-4, 'wide clamp' = +-12.

Why we care

Some subjects get almost everything wrong (one deepseek run scored 0.8%). Their true theta is very negative.
If the prior says 'that is nearly impossible', it keeps pulling them back toward the middle, and we predict they will do better than they do.

How hard does each prior pull?

The prior acts like a rubber band tied to zero. The further the subject's theta is from zero, the more the band pulls it back.
The question is how the pull changes as theta gets more extreme.



What the curves say

Gaussian (blue)
The pull keeps growing the further out you go. At theta = 12 it is tugging 3 units hard. Evidence has to shout louder and louder to move it.

Student-t (orange, red)
The pull rises, peaks, then FADES. Far out, the band goes slack and the answers decide. Cauchy fades the fastest.

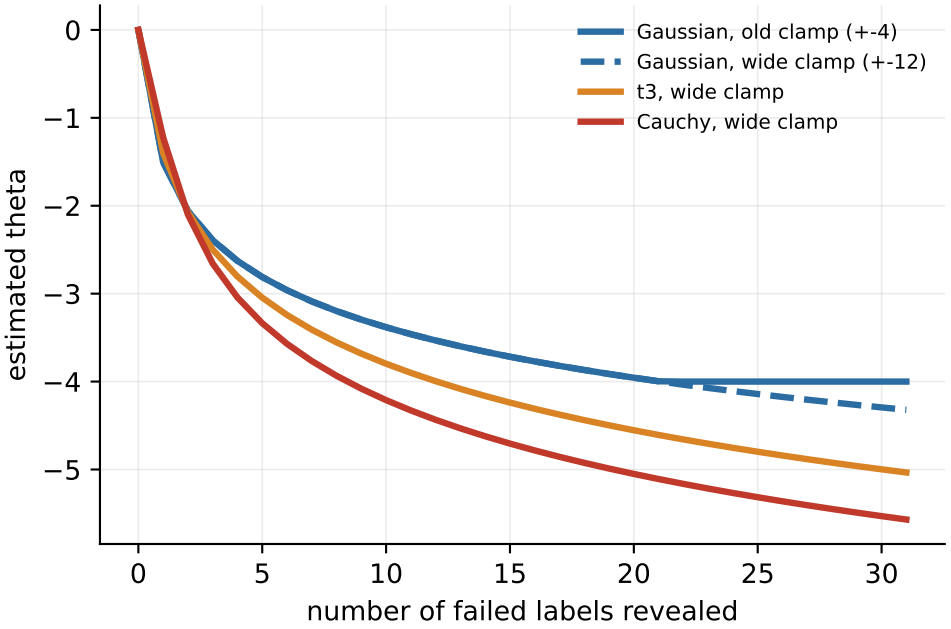
In plain words

- Ordinary subjects (theta near 0): all three priors pull about the same. No difference.
- Extreme subjects: the Gaussian keeps dragging them back. The heavy-tailed priors let go.
- That is the whole idea: shrink small, uncertain estimates, but do not crush a subject whose labels clearly say 'always fails'.
- Trade-off: letting go also means a few unlucky labels can push a normal subject too far.
  That is why we also tried the middle option (t3) and not only Cauchy.

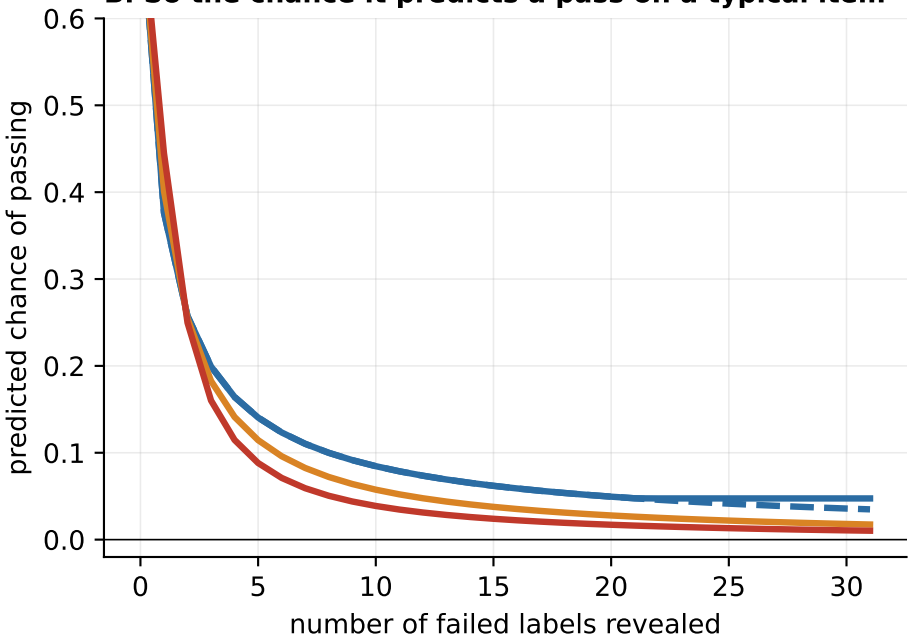
A worked example: a subject that fails everything

We reveal labels one by one. Every label is a FAIL, on items that most subjects pass (easy items).
The plots below use the real solver from our model, so this is what the code would do, not a cartoon.

A. What the solver believes theta is



B. So the chance it predicts a pass on a typical item



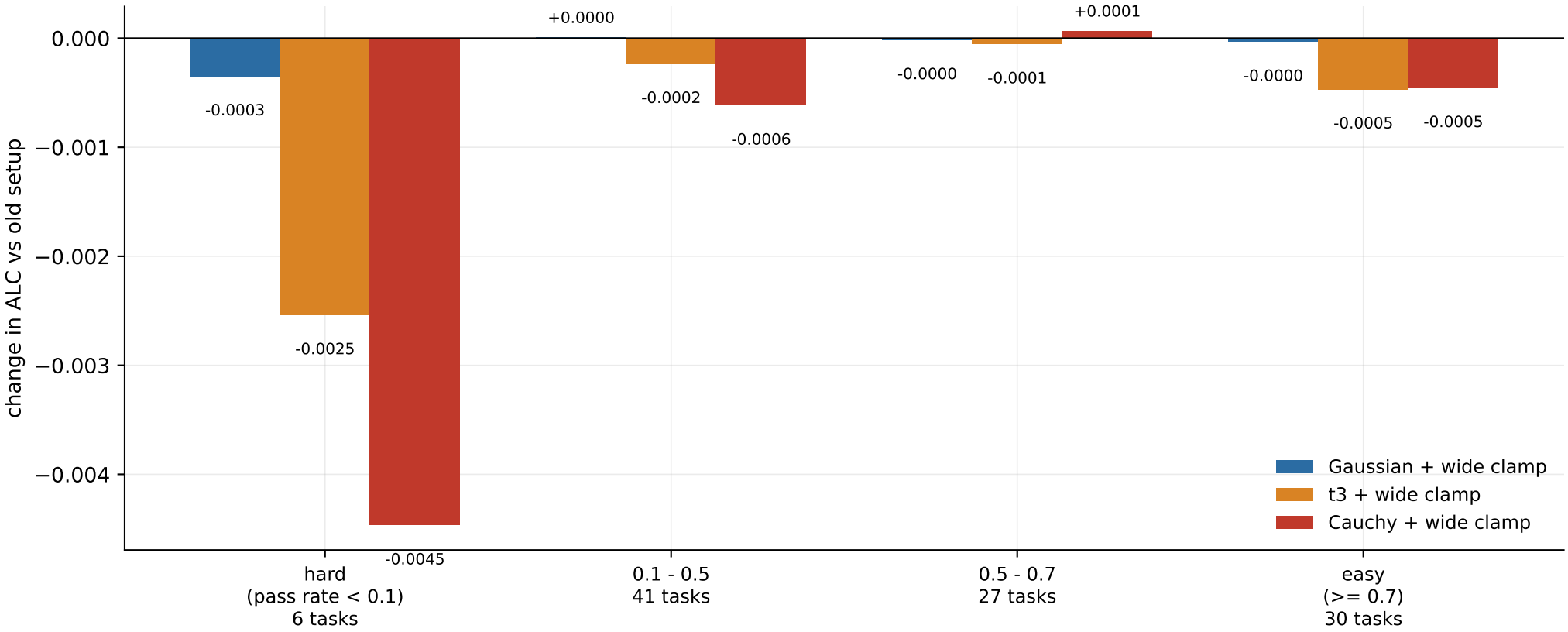
What to notice

- After 31 straight failures: Gaussian with the old clamp says theta = -4.0, Gaussian with the wide clamp -4.3, t3 says -5.0, Cauchy says -5.6.
- The flat part of the solid blue line at -4 is the CLAMP (a wall), not the prior.
- Remove the wall (dashed blue) and the Gaussian still slows down a lot: its own pull keeps growing, so it only creeps lower. Widening the clamp alone changes little. That matches our result.
- The heavy-tailed priors keep sliding down as the failures pile up, so the predicted pass chance keeps shrinking, as it should.
- At 0 labels all four are identical: with no evidence, the prior is the guess.

Setup: items of difficulty -1 (an average subject passes about 73%), one ability dimension, prior width 2 (variance 4).

Did it help? Measured results

Change in Brier ALC against the old setup (Gaussian, +-4 clamp). Lower is better, so BARS GOING DOWN = BETTER.
Tasks are grouped by how often the subject passes. Model: me5 embeddings + 4-dimensional ability. Validation set, one seed.



What it says

- Widening the clamp alone (blue) does almost nothing. The prior is the problem, not the wall.
- Heavy tails help the hard tasks a lot: Cauchy improves them by about 0.0045, t3 by about 0.0025.
- The easy tasks are not hurt after calibration (all within 0.0005 of the old setup).
- Overall the gain is small, about 0.0004 to 0.0006, because only 6 of 104 tasks are hard.

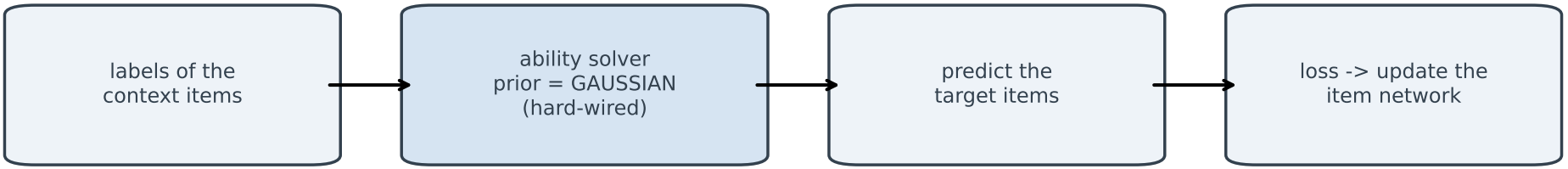
Be careful

- One seed and only 6 hard tasks: a lead, not a proven result.
- Before calibration, Cauchy made the easy tasks worse (+0.0038). It looks fine only after calibration, so t3 is the safer pick.

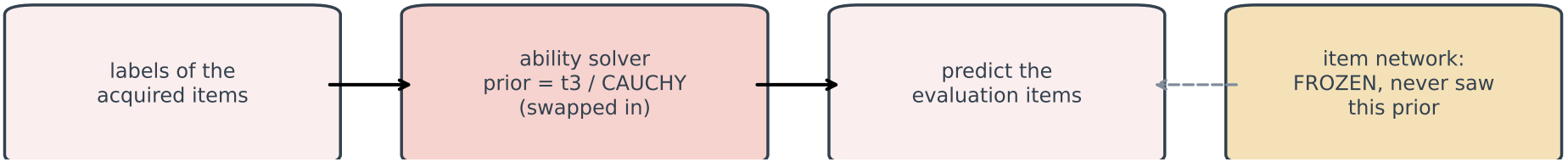
The mismatch: one prior in training, another at test time

Our model runs the same ability solver (Newton MAP) in two places. Only one of them got the new prior.

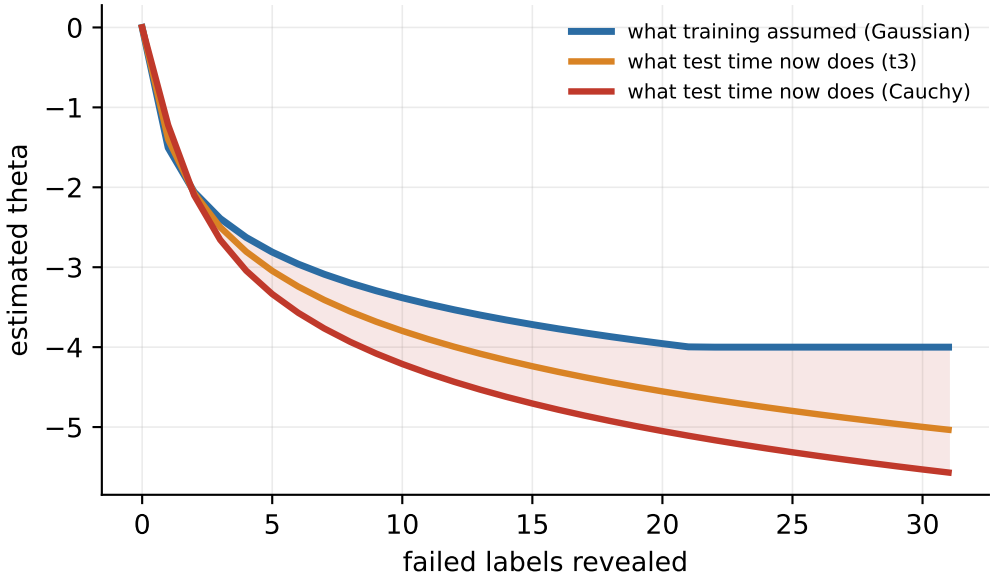
TRAINING (item network learns difficulty and discrimination)



TEST TIME (what we changed)



The shaded gap is the mismatch



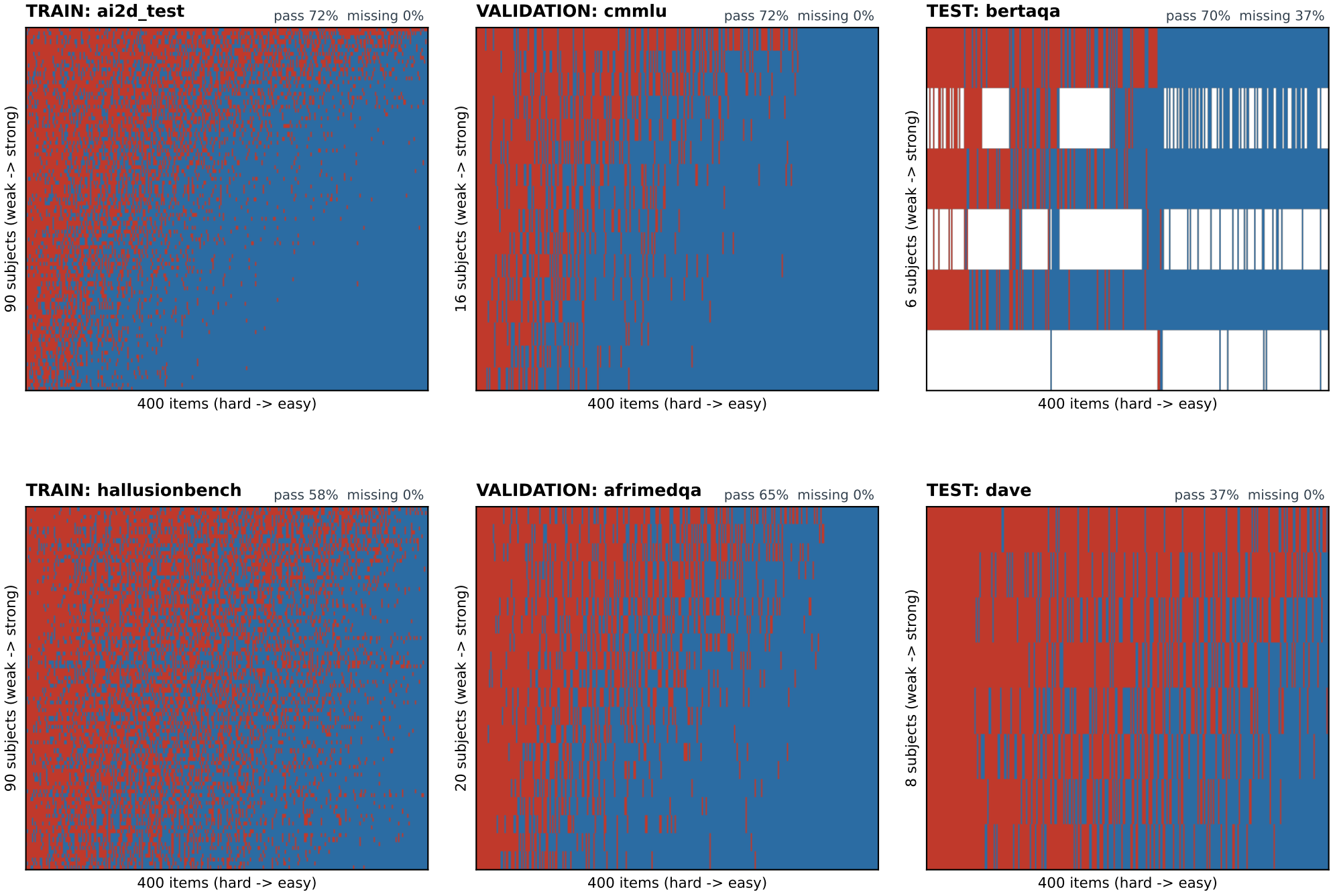
Why it matters

- The item network was trained while the Gaussian kept shrinking theta. It may have learned difficulties that quietly make up for that shrinkage.
- At test time we remove the shrinkage, so those difficulties are slightly off.
- Even so, heavy tails helped the hard tasks (one seed, 6 tasks). Retraining with the new prior MIGHT help more, but we have not tested it.

Not done yet: retraining with the t prior in the training loop (needs a GPU run).

The raw response matrix, by role

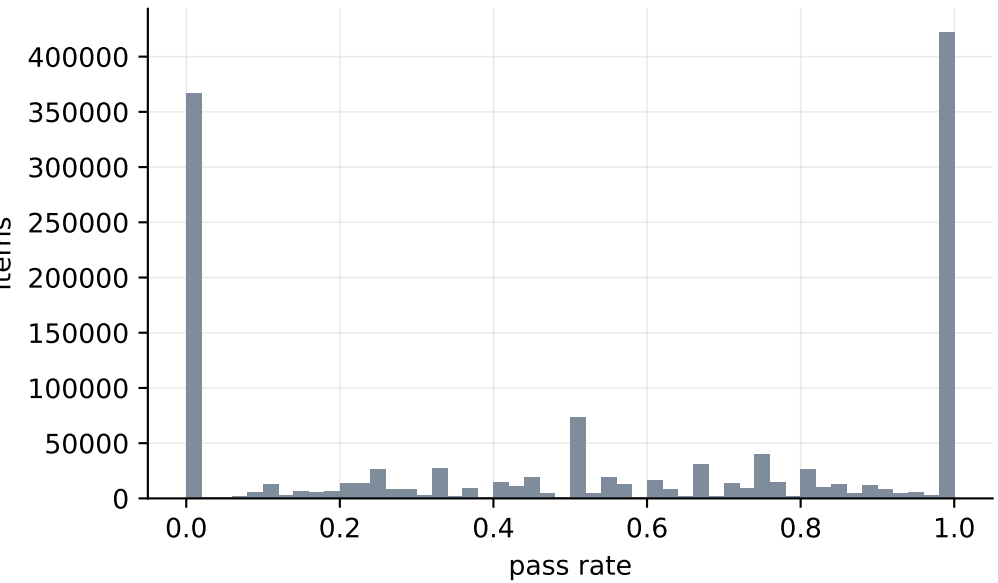
Each panel is one benchmark: rows = subjects (weakest at the top, strongest at the bottom), columns = items (hardest on the left). Blue = correct, red = wrong, white =
Subjects with the most responses and items answered by the most subjects are shown. Roles are ours: train / validation / test benchmarks.



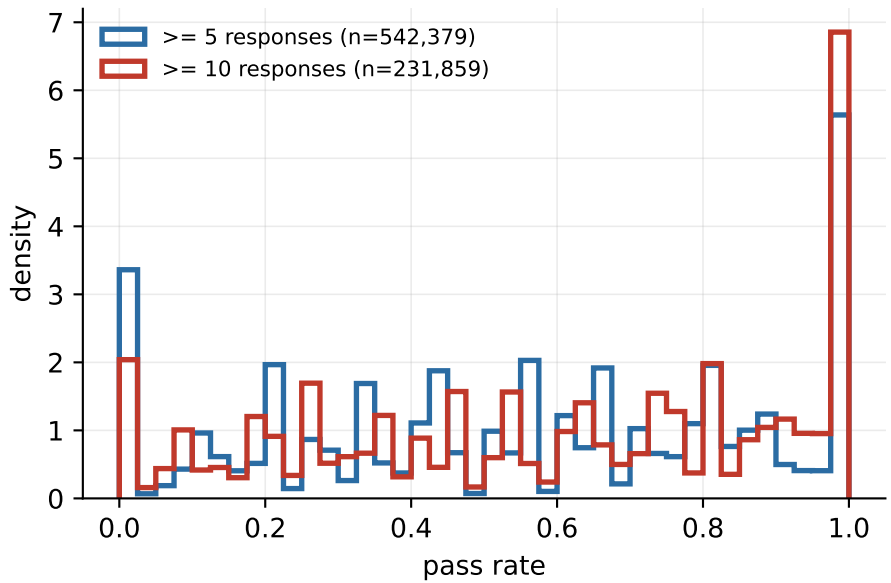
Item pass rate: A and B, and the same thing per role

Pass rate = share of responses that were correct (low = hard). A and B are the global plots from the data analysis; C and D split the items by our train / validation / test ro

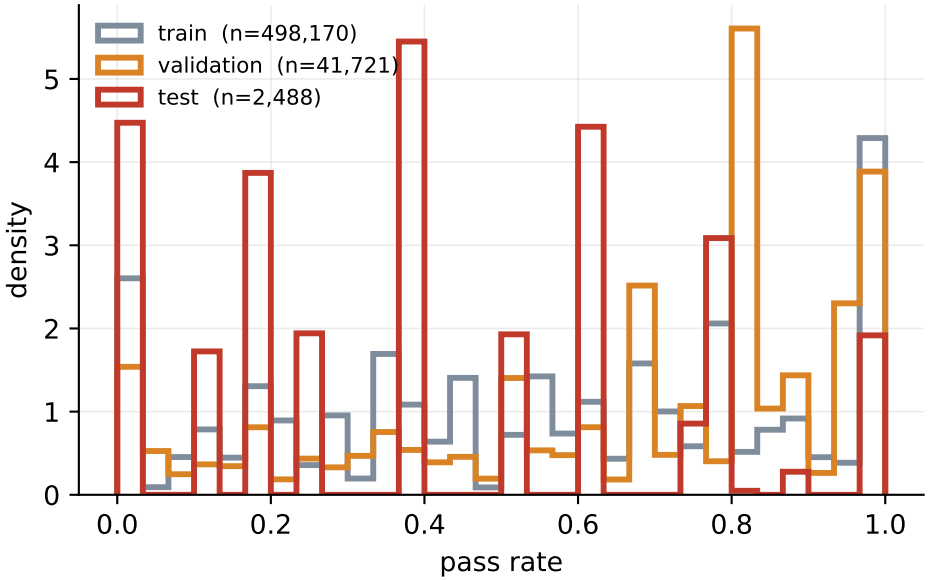
A. All items (spikes = single-response items)



B. Items with enough responses to trust



C. Items with >= 5 responses, by role



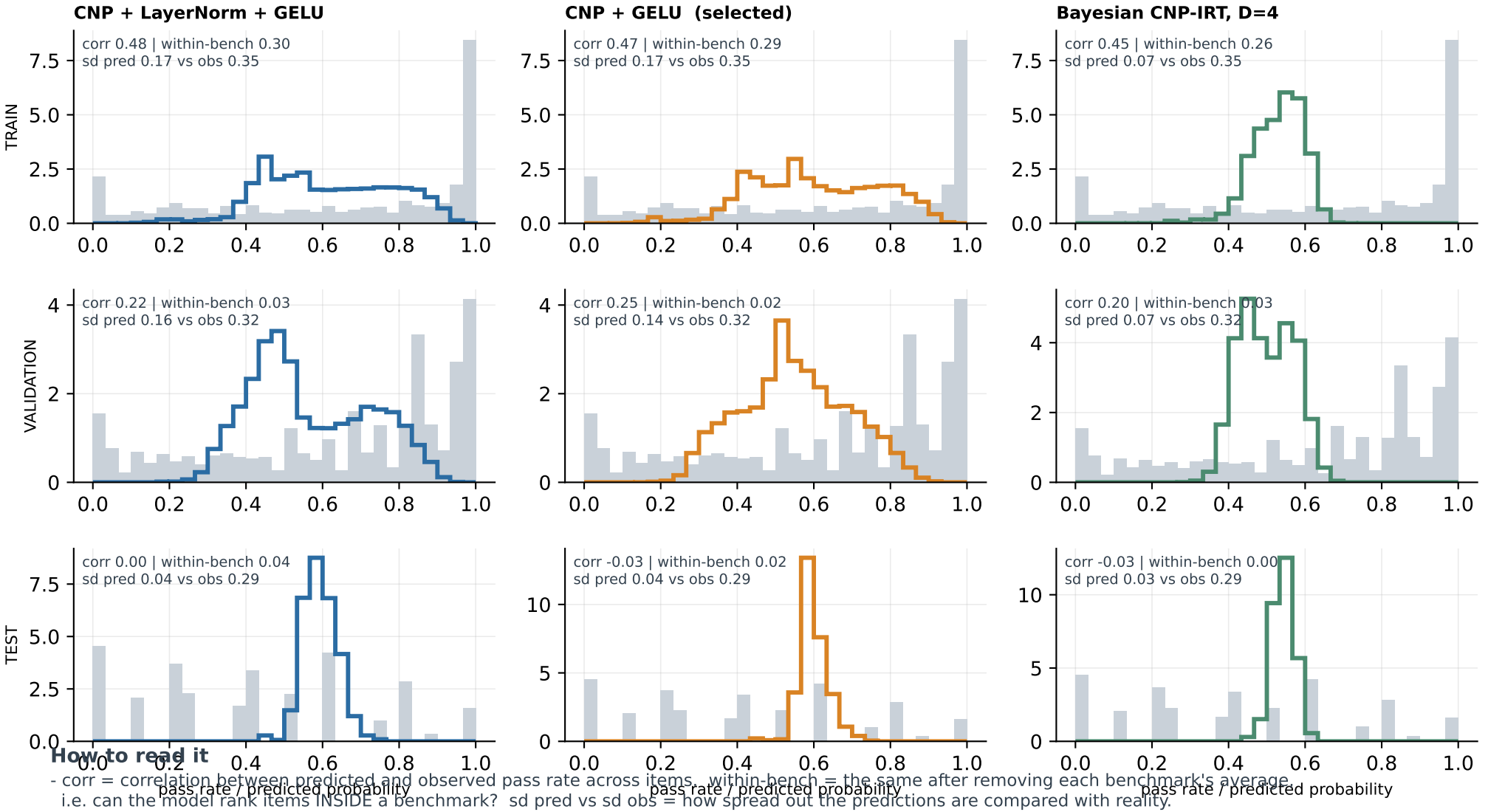
D. Per role

role	items	>=5 resp	single	mean	sd
train	1,180,749	498,170	38%	0.54	0.32
validation	67,510	41,721	15%	0.66	0.30
test	90,463	2,488	55%	0.42	0.29

single = share of items answered once. mean / sd are over items with >= 5 responses. Validation has only 5 benchmarks and test 6, so their curves are lumpy: one benchmark can dominate a role.

Is the model learning the item pass-rate distribution?

Grey = observed pass rate of items that have >= 5 responses. Coloured line = what the model predicts for the same items with NO labels revealed (budget 0), averaged
If the model had learned item difficulty, the coloured line would follow the grey shape. Predictions are raw, not calibrated.



How to read it

- corr = correlation between predicted and observed pass rate across items. within-bench = the same after removing each benchmark's average, i.e. can the model rank items INSIDE a benchmark? sd pred vs sd obs = how spread out the predictions are compared with reality.
- A model that only knows 'this benchmark is easy' gets a decent corr but a within-bench corr near 0, and predictions bunched up in the middle.

Predicted vs observed pass rate, item by item

Each point is an item with >= 5 responses. The red dashed line is perfect agreement. Top row: no labels revealed. Bottom row: 31 labels revealed (order 0), which lets the
Validation and test items only (those are the roles with labels revealed).

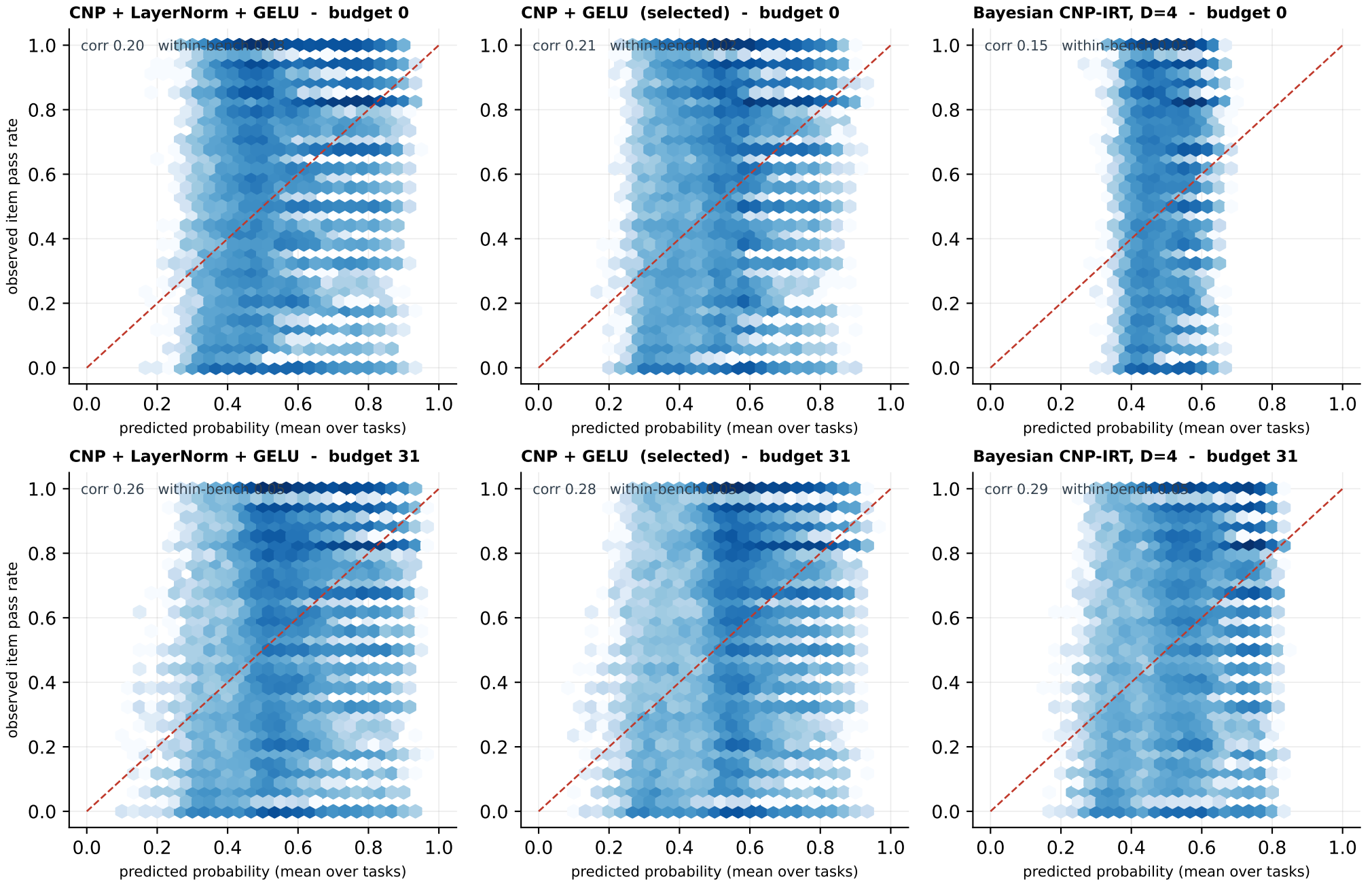


Figure 11 analog: how well does a few-label estimate recover the truth? (validation benchmarl

For each subject (task) and each random label order we estimate its accuracy two ways: CTT = the plain average of the labels revealed so far (grey);
the model = the mean probability it predicts over the whole evaluation pool. Plots show estimate MINUS the true evaluation-pool accuracy. A good estimator is centred o

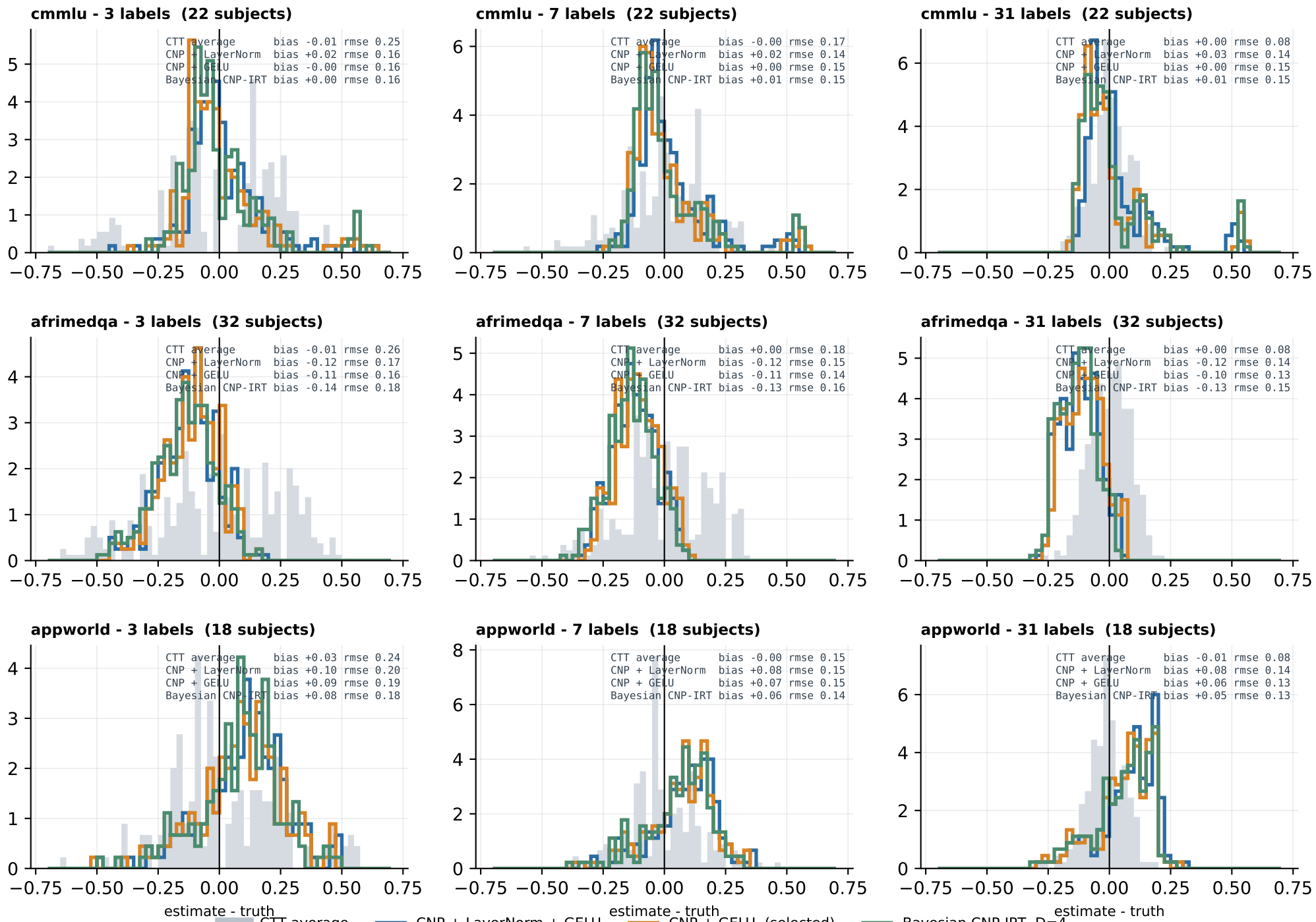


Figure 11 analog: how well does a few-label estimate recover the truth? (test benchmarks)

For each subject (task) and each random label order we estimate its accuracy two ways: CTT = the plain average of the labels revealed so far (grey);
the model = the mean probability it predicts over the whole evaluation pool. Plots show estimate MINUS the true evaluation-pool accuracy. A good estimator is centred o

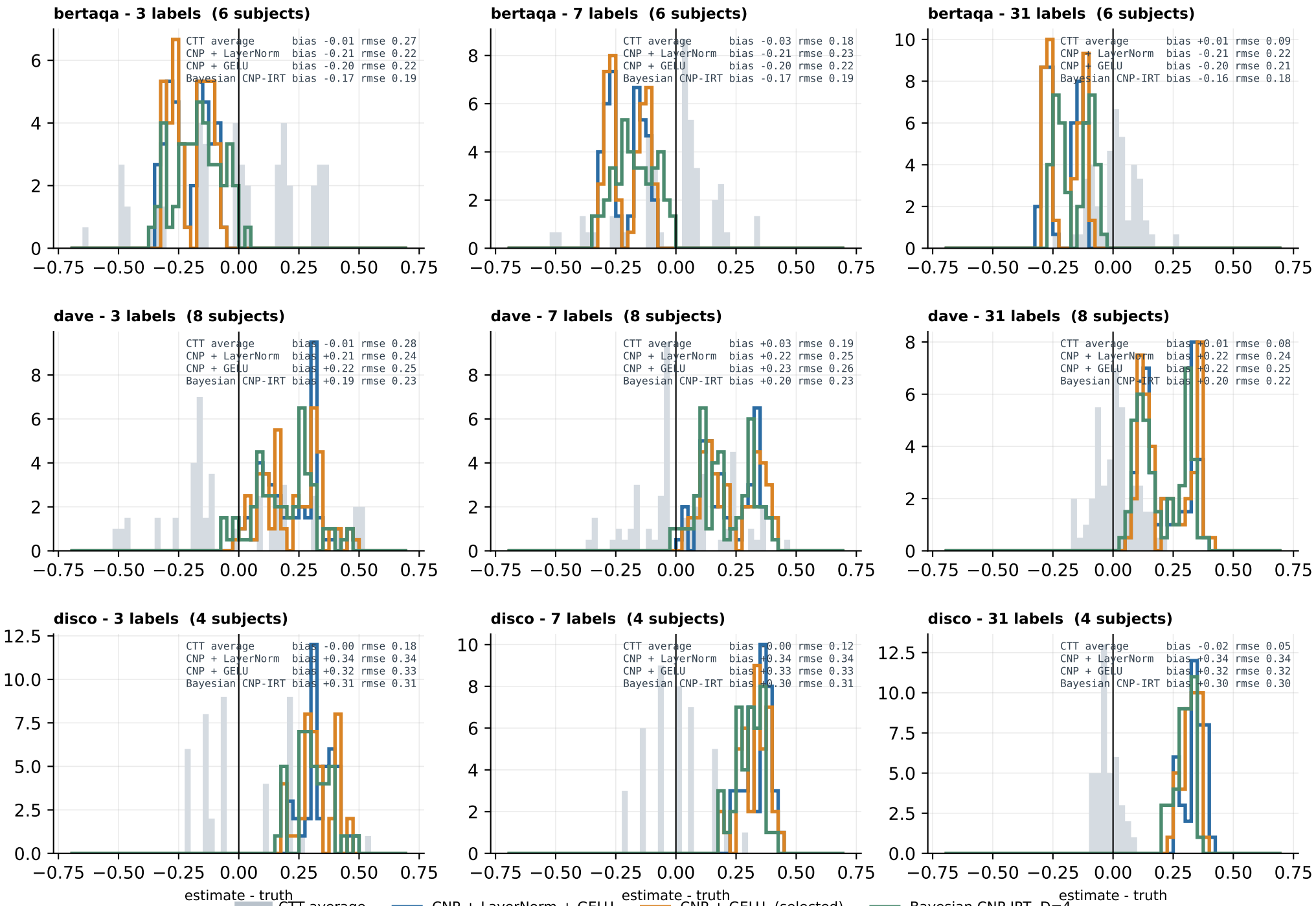
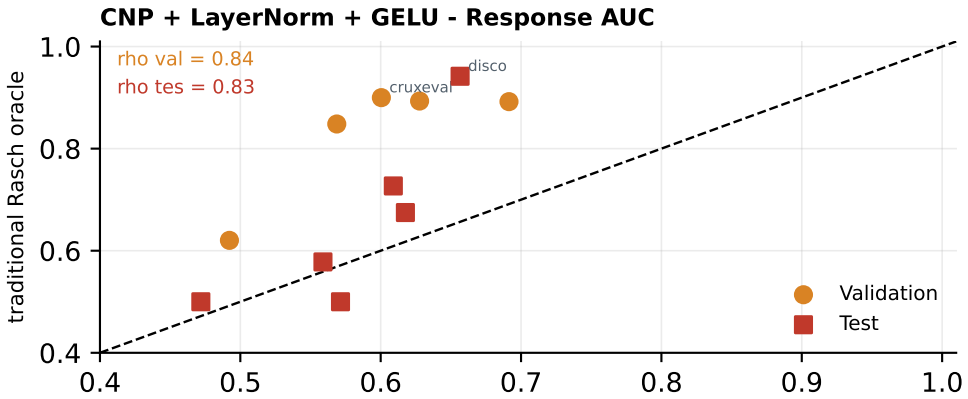
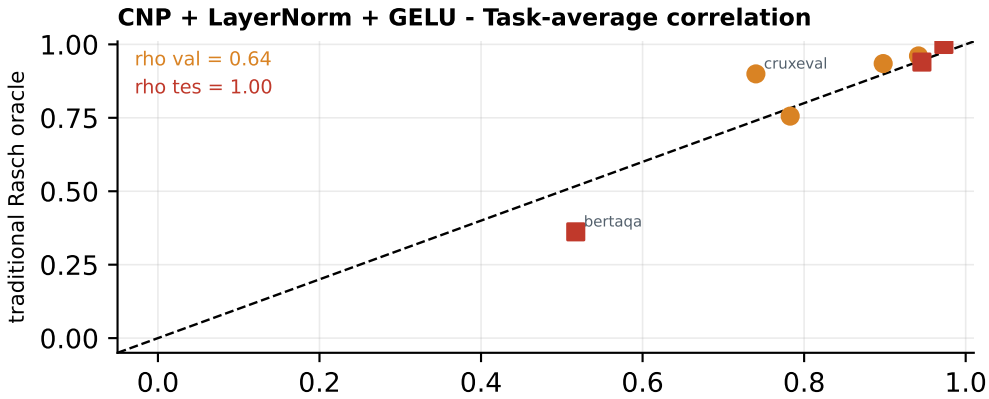
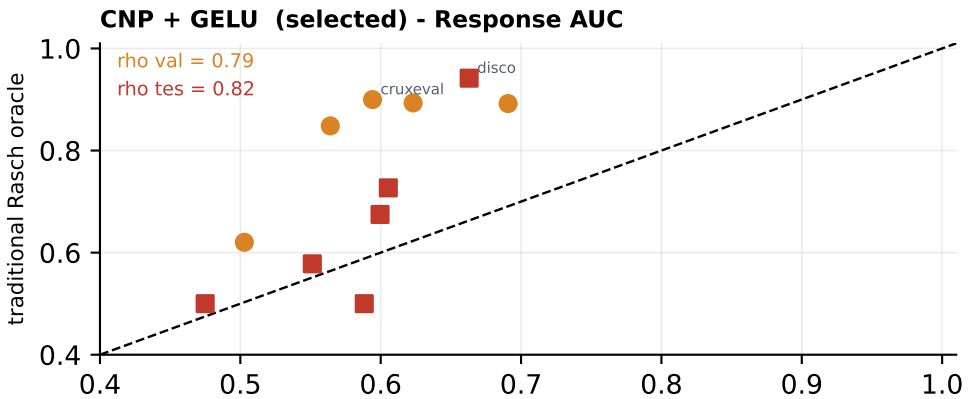


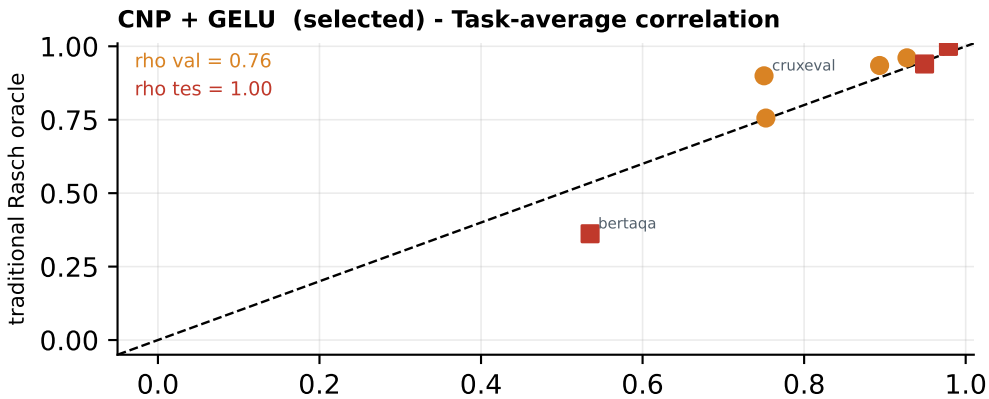
Figure 4 analog: neural predictions vs response-calibrated Rasch

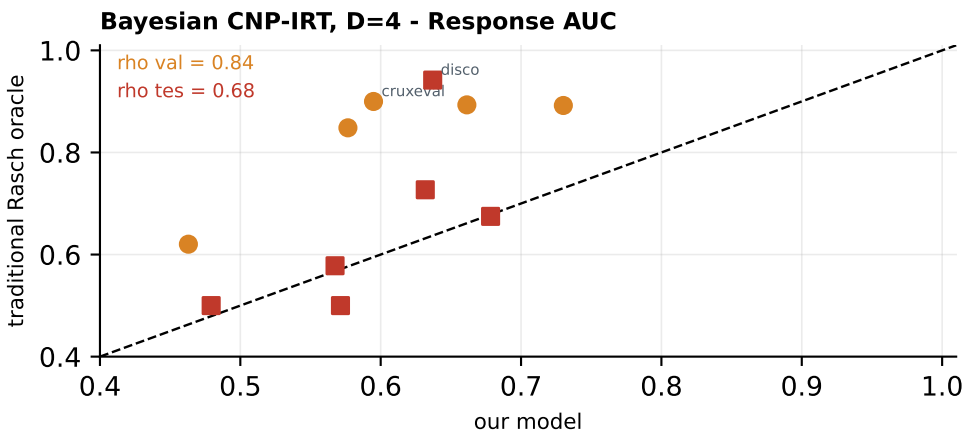
Each point is one held-out benchmark at budget 31. x = our model; y = a traditional scalar Rasch oracle whose item logits come from other subjects' raw responses. Both
Left: response AUC. Right: correlation across tasks between predicted and observed mean accuracy. Dashed line = equal fit.

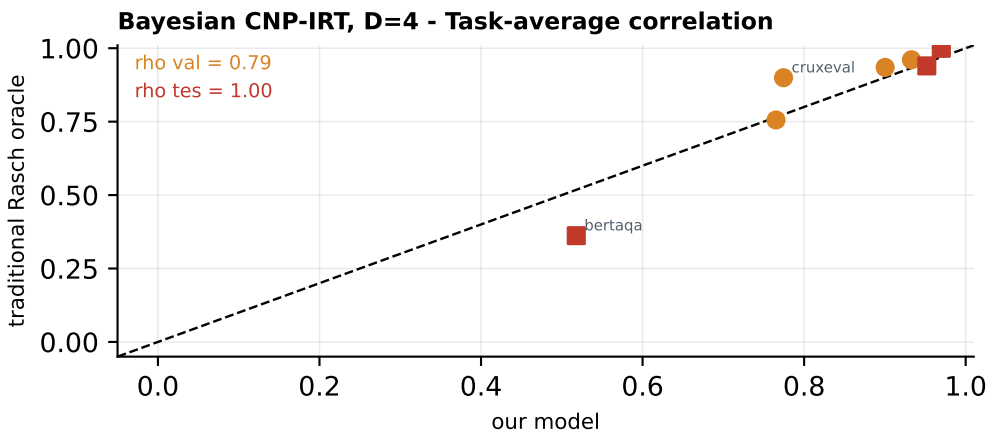










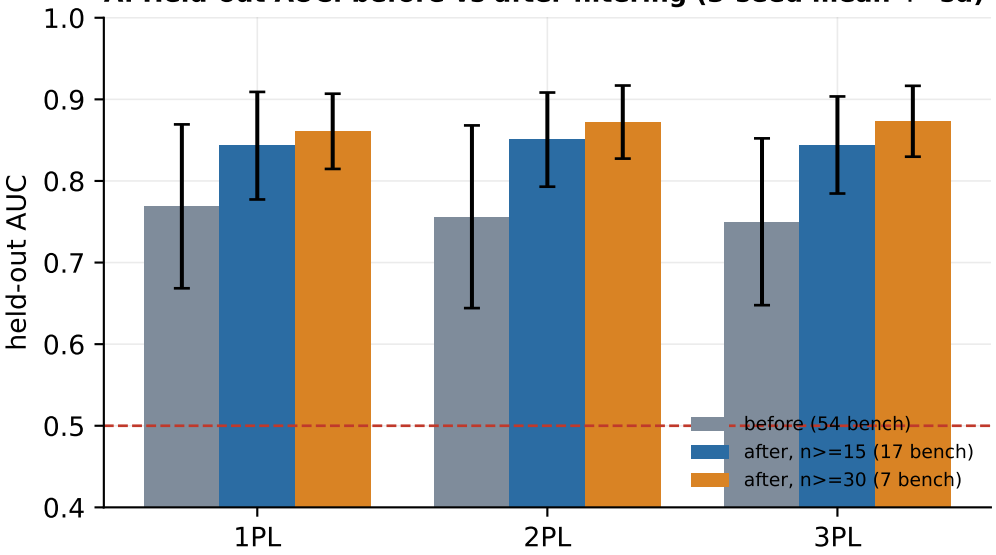


Important: this is our competition-aligned analog, not the paper's identical split. Validation/test denote held-out benchmark roles. Item logits use leave-one-subject-out counts with Beta(1,1) smo

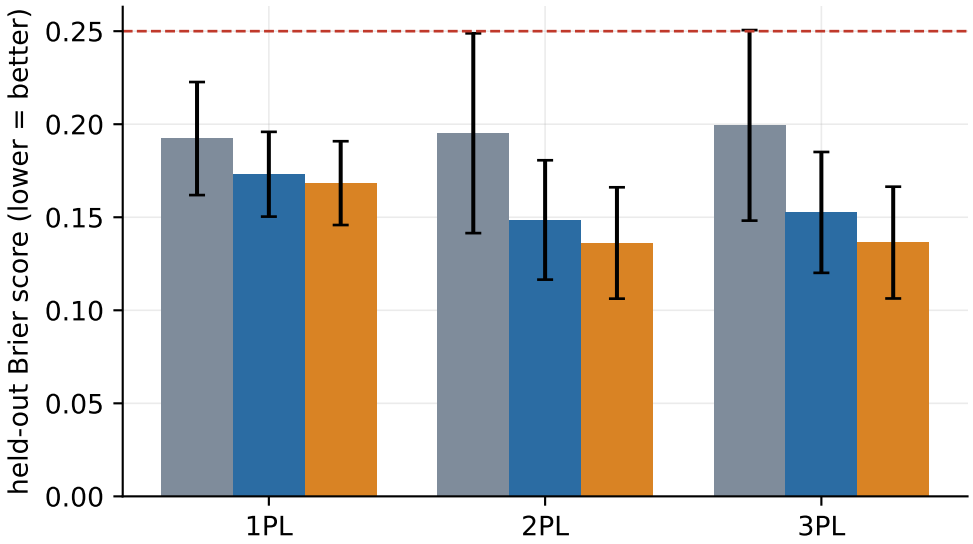
Does filtering let classic IRT (no text) recover item difficulty?

Plain 1PL/2PL/3PL: theta (per subject) and item params are free numbers, jointly
MLE-fit by Adam on the response matrix -- no embeddings, no cold start. Filters: dedupe content, drop zero-variance items,
drop items with <n subjects, drop subjects with <80 items/task. n=15 keeps 17 benchmarks, n=30 keeps 7. 3 seeds, 20% cell holdout.

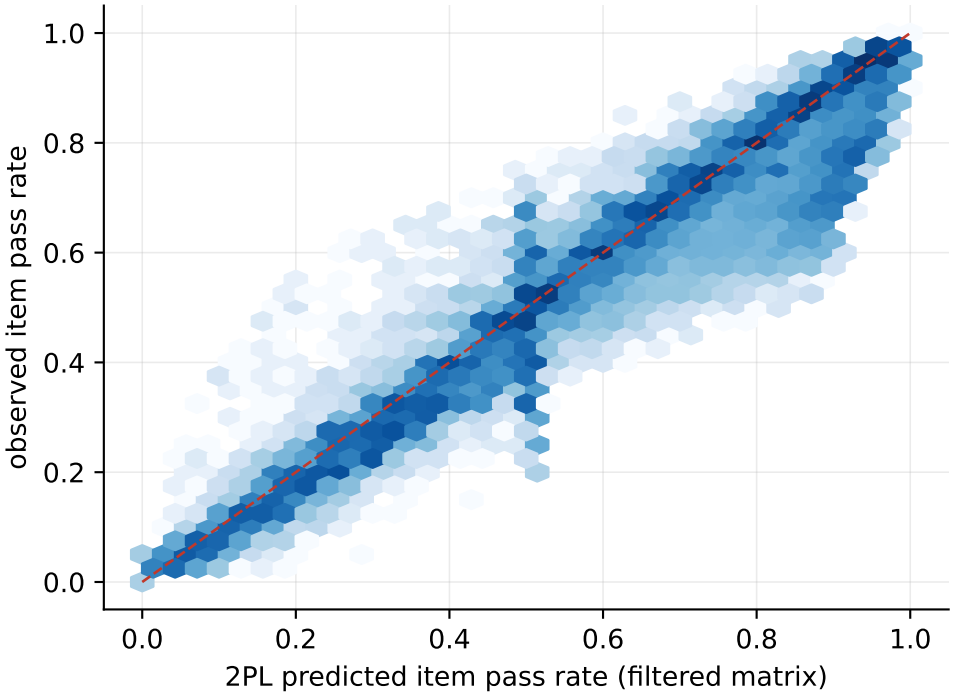
A. Held-out AUC: before vs after filtering (3-seed mean +- sd)



B. Held-out Brier: before vs after filtering



C. Item-level fit, n>=15 filter: corr 0.98, within-bench 0.98 (n=122,973)



D. Within-benchmark correlation, side by side

Classic 2PL, filtered (this page) (r = 0.98)

CNP + GELU, neural, validation (r = 0.02)

CNP + LN + GELU, neural, validation (r = 0.03)

Bayesian CNP-IRT, neural, validation (r = 0.03)

Classic IRT with no text and the n>=15 filtered matrix recovers item difficulty almost perfectly within a benchmark (0.98). The neural models, fit on the FULL unfiltered data with text embeddings, recover almost nothing (0.02-0.03). The gap is not about text vs no text -- it is the data the model was allowed to learn from.

n>=30 (paper's threshold) fits even better -- 2PL AUC 0.872, Brier 0.136 -- but keeps only 7 of 54 benchmarks, against 17 at n>=15. Quality vs coverage: the stricter filter fits better on what remains.

How different are the benchmark-holdout train, validation, and test distributions?

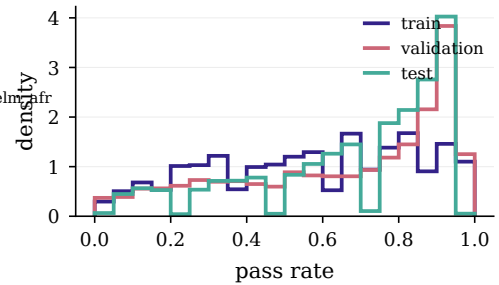
Full leakage-safe filtered split before panel sampling. Different colours are disjoint benchmark sets; histogram total-variation distance (TVD) is relative to train.

Split sizes and item-pass-rate shift

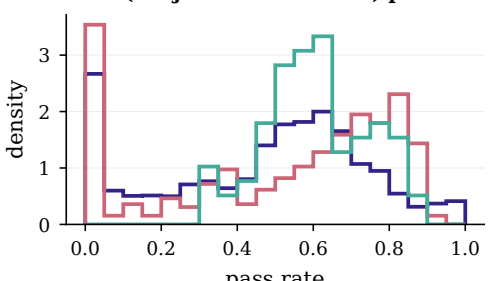
	benchmar	cells	tasks	items	mean y	sd item	TVD item p
train	11 g, appworld, brace, br	5,359,69	6,013	109,987	0.512	0.267	0.000
validation	3 ai2d_test, bird_sql, f	812,782	390	3,451	0.676	0.280	0.216
test	3 algotune, cmmlu,	166,450	78	9,597	0.663	0.253	0.315

ach, helm, ifr

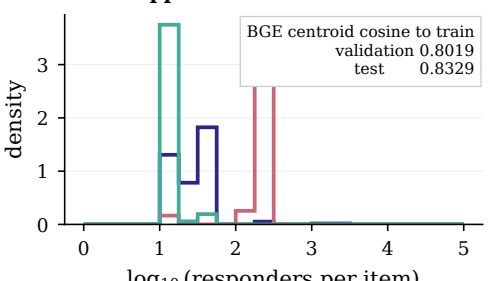
Item pass rate



Task (subject × benchmark) pass rate



Item support



Is the Neural Process learning the panel item pass-rate distribution?

Budget 0 only. Grey is empirical pass rate over the frozen validation evaluation cells; colour is the model probability, averaged over tasks containing the item.

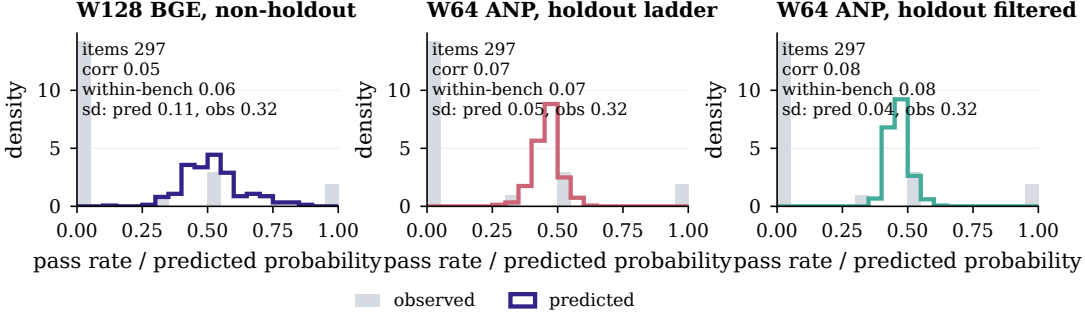
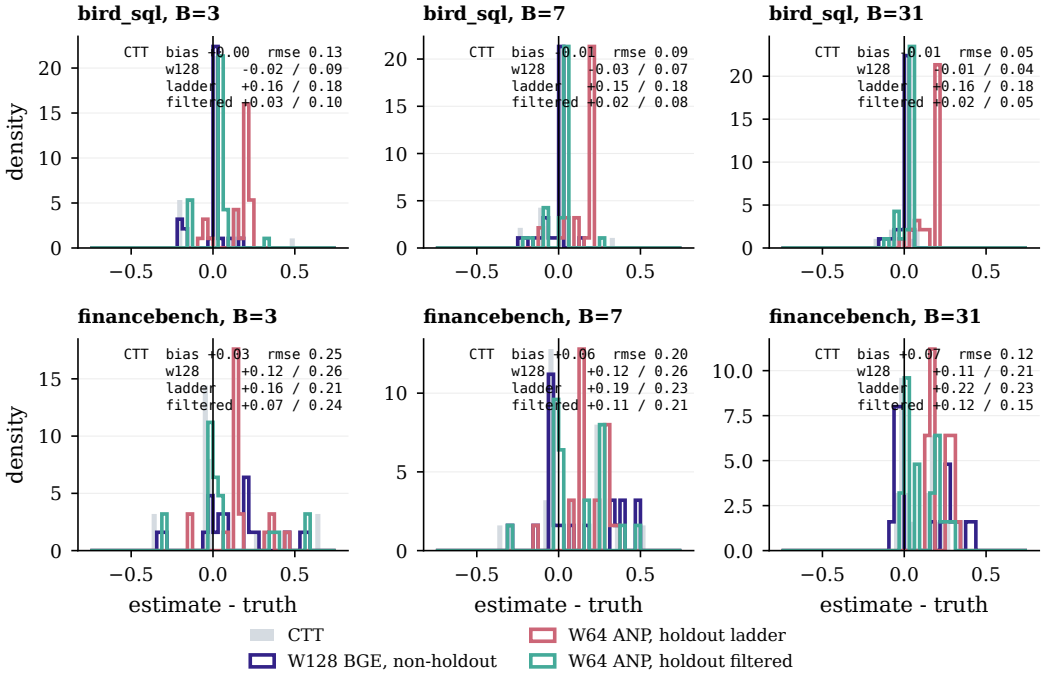


Figure 11 analog: does a few-label estimate recover evaluation-pool accuracy?

Random streaming acquisition only. Each observation is one task and candidate order. Curves show estimate minus that task's held-out evaluation-pool accuracy.



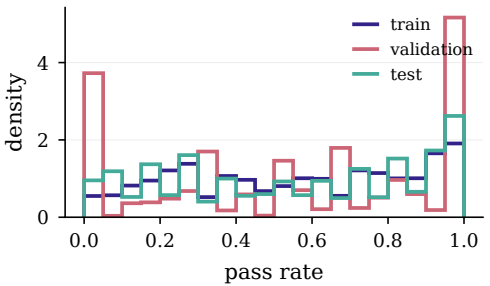
How different are splits in the original non-benchmark-holdout NP training?

Exact standard NP split: random subject × benchmark tasks, plus a global 20% held-item set removed from train/validation. Test below retains all task items for the later response/item evaluation axes.

Split sizes and pass-rate shift

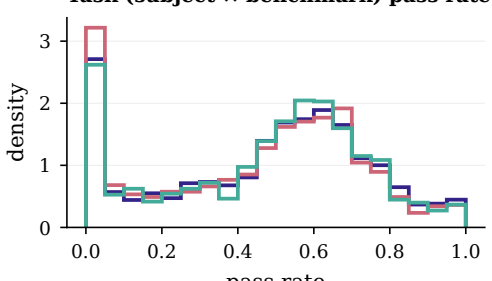
	# benchma	cells	tasks	items	mean y	d item	TVD item p
train	7	2,620,41	4,073	45,047	0.529	0.296	0.000
validation	7	552,360	939	45,047	0.502	0.369	0.475
test	7	1,174,38	1,252	56,308	0.512	0.325	0.156

Item pass rate

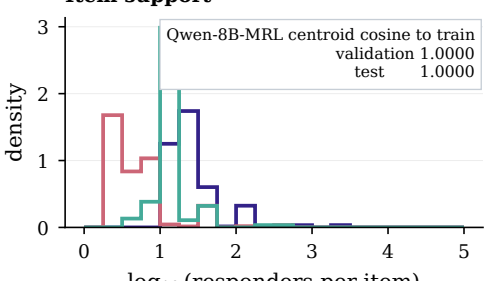


Overlap with train (benchmark fraction / item Jaccard)
validation: 1.00 / 1.00
test: 1.00 / 0.80

Task (subject × benchmark) pass rate



Item support



Is ANP-128 with Qwen embeddings learning the item pass-rate distribution?

Original non-benchmark-holdout split, budget 0. Grey is observed pass rate among at least five responses in that role; colour is the ANP prior prediction with an empty context. These are diagnostic predictions, not calibrated probabilities.

